

系统设计面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer

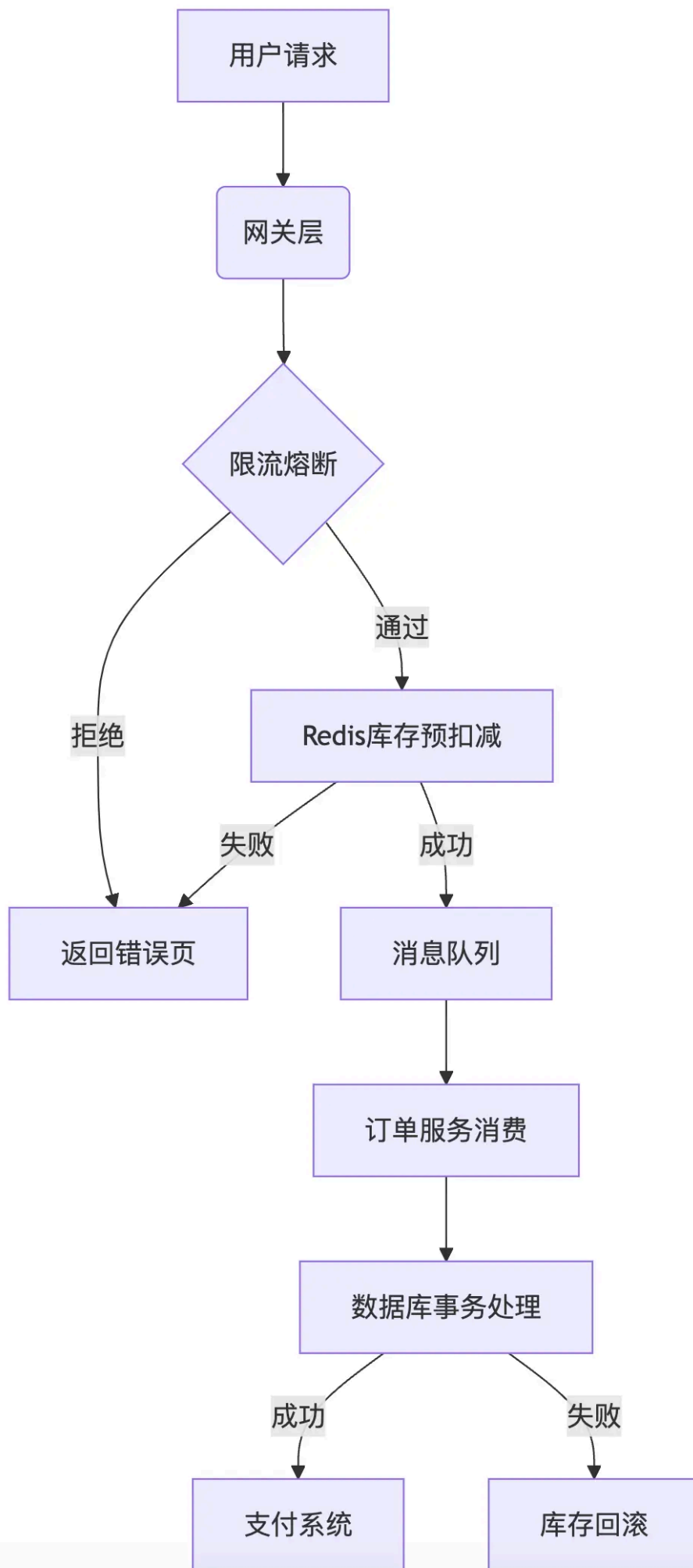


如何设计一个秒杀场景？

秒杀场景的核心特点是**高并发**、**低库存**、**短时间爆发式访问**。因此，设计时需要解决以下几个问题：

1. **高并发处理**：如何应对大量用户同时访问？
2. **库存一致性**：如何保证库存不会超卖或少卖？
3. **用户体验**：如何减少用户等待时间，避免页面崩溃？
4. **防刷机制**：如何防止恶意用户利用脚本抢购商品？

面对上面这些问题，可以针对每一层做一些设计：



1、前端层：

- **静态资源分离**：将秒杀页面的静态资源（如HTML、CSS、JS）部署到CDN（内容分发网络），减轻服务器压力。
- **请求拦截**：活动未开始时，前端按钮置灰；通过验证码、点击频率限制。

2、网关层：

- **流量拦截**：使用API网关对请求进行初步过滤，例如IP限流、黑名单拦截等。
- **身份验证**：通过Token或签名验证用户身份，防止未登录用户直接访问秒杀接口。

3、缓存层：

- **Redis缓存库存**：将商品库存信息存储在Redis中，利用其高性能特性处理库存扣减操作。
- **预热数据**：在秒杀活动开始前，将商品信息和库存数据加载到Redis中，减少数据库压力。

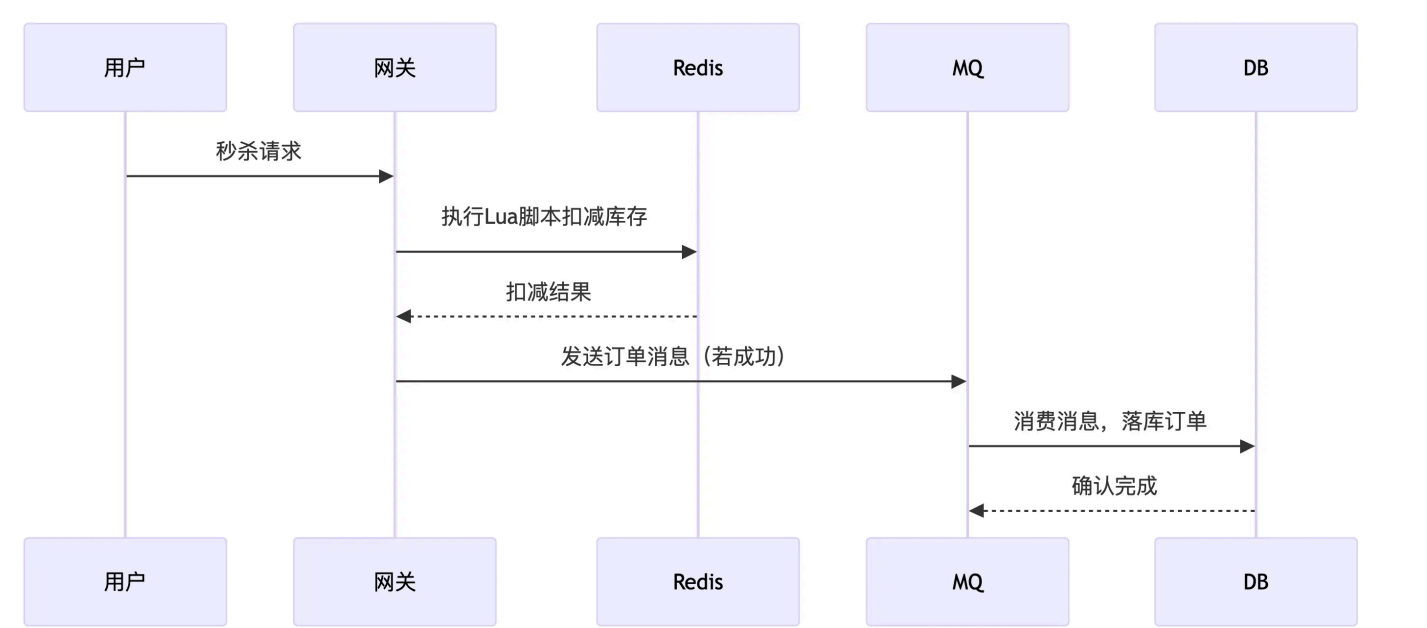
4、消息队列：

- **削峰填谷**：使用消息队列（如Kafka）将用户的秒杀请求异步化，避免直接冲击后端服务。
- **订单处理**：将成功的秒杀请求放入队列，由后台服务异步生成订单，提高系统吞吐量。

5、数据库层

- **乐观锁**：在库存扣减时，使用乐观锁确保库存一致性。
- **读写分离**：通过主从复制实现数据库的读写分离，提升查询性能。

关键的核心业务逻辑实现， **库存防超卖方案采用：Redis原子操作 + 异步扣减数据库。**



具体流程如下：

- **秒杀请求达到**：用户发起秒杀请求，系统接收到请求后，首先进行一些基础校验（如用户身份验证、活动是否开始等）。如果校验通过，进入库存扣减逻辑。
- **Redis库存扣减**：在Redis中检查商品库存是否充足。例如，使用 `GET` 命令获取当前库存数量。如果库存不足，直接返回失败，结束流程。如果库存充足，使用Redis的原子操作（如 `DECR` 或Lua脚本）扣减库存。
- **异步更新数据库**：如果Redis库存扣减成功，生成一个秒杀成功的消息，并将其放入消息队列
- **后台服务消费消息**：后台服务从消息队列中消费秒杀成功的消息，执行以下操作：
 - 1、为用户创建订单记录；
 - 2、使用乐观锁将数据库中的库存数量减少1；
 - 3、通过唯一标识（如用户ID+商品ID+时间戳）防止重复消费。
- **最终一致性校验**：在Redis库存扣减和数据库库存更新之间，可能会存在短暂的不一致状态。为了保证最终一致性，可以采取以下措施：
 - 1、定期将Redis中的库存数据与数据库进行同步。
 - 2、如果发现Redis和数据库库存不一致，触发补偿逻辑（如回滚订单或调整库存）。

设计题：订单到了半个小时，半个小时未支付就取消

有多种实现订单超时自动取消的技术方案，包括定时轮询、JDK的延迟队列、时间轮算法、Redis实现以及MQ消息队列中的延迟队列和死信队列。

- 定时轮询：基于SpringBoot的Scheduled实现，通过定时任务扫描数据库中的订单。优点是实现简单直接，但缺点是会给数据库带来持续压力，处理效率受任务执行间隔影响较大，且在高并发场景下可能引发并发问题和资源浪费。
- JDK的延迟队列（DelayQueue）：基于优先级队列实现，减少数据库访问，提供高效的任务处理。优点是内部数据结构高效，线程安全。缺点是所有待处理订单需保留在内存中，可能导致内存消耗大，且无持久化机制，系统崩溃时可能丢失数据。
- 时间轮算法：通过时间轮结构实现定时任务调度，能高效处理大量定时任务，提供精确的超时控制。优点是实现简单，执行效率高，且有成熟实现库。缺点同样是内存占用和崩溃时数据丢失的问题。
- Redis实现：
 - 有序集合（Sorted Set）：利用有序集合的特性，定时轮询查找已超时的任务。优点是查询效率高，适用于分布式环境，减少数据库压力。缺点是依赖定时任务执行频率，处理超时订单的实时性受限，且在处理事务一致性方面需要额外努力。
 - Key过期监听：利用Redis键过期事件自动触发订单取消逻辑。优点是实时性好，资源消耗少，支持高并发。缺点是对Redis服务的依赖性强，极端情况下处理能力可能成为瓶颈，且键过期有

一定的不确定性。

- MQ消息队列：
 - 延迟队列（如RabbitMQ的`rabbitmq_delayed_message_exchange`插件）：实现消息在指定延迟后送达处理队列。优点是处理高效，异步执行，易于扩展，模块化程度高。缺点是高度依赖消息队列服务，配置复杂度增加，可能涉及消息丢失或延迟风险，以及消息队列与数据库操作一致性问题。
 - 死信队列：通过设置队列TTL将超时订单转为死信，由监听死信队列的消费者处理。优点是能捕获并隔离异常消息，实现业务逻辑分离，资源保护良好，方便追踪和分析问题。缺点是相比延迟队列，处理超时不够精确，配置复杂，且同样存在消息处理完整性及一致性问题。

不同方案各有优劣，实际应用中应根据系统的具体需求、资源状况以及技术栈等因素综合评估，选择最适合的方案。在许多现代大型系统中，通常会选择消息队列的延迟队列或死信队列方案，以充分利用其异步处理、资源优化和扩展性方面的优势。

如果做一个大流量的网站，单Redis无法承压了如何解决？

- 读写分离：部署多个 Redis 从节点（Slave），主节点（Master）负责写操作，从节点负责读操作。主节点将数据同步到从节点，从节点可以处理大量的读请求，减轻主节点的压力。
- 构建集群：部署 Redis Cluster 集群，Redis Cluster 将数据自动划分为 16384 个槽（slots），每个槽都可以存储键值对。这些槽会被分配到多个 Redis 节点上，通过哈希函数将键映射到相应的槽，再由槽映射到具体的 Redis 节点。例如，使用 $\text{CRC16}(\text{key}) \% 16384$ 来确定键属于哪个槽，然后根据槽与节点的映射关系将键值对存储到相应节点。通过数据分片，将数据和请求分散到多个节点，避免单个节点的负载过高。不同节点负责不同的槽，各自处理一部分请求，实现负载均衡。

如何设计一个可重入的分布式锁，用什么结构设计？

可重入锁允许同一线程在持有锁的情况下多次获得锁而不会产生死锁。当线程请求锁时，如果它已经拥有了该锁，则可以直接获得锁，锁的计数器会增加。在释放锁时，计数器会减少，只有当计数器为零时，锁才会真正释放。

在分布式系统中，通常会使用诸如 Redis、Zookeeper 或 etcd 等组件来实现锁的管理。下面以 Redis 为例，简单描述如何设计一个可重入的分布式锁。

设计要素：

- **锁的标识符（Lock ID）**：用于标识锁的唯一性。

- **持有者标识 (Owner ID)**：线程或进程的唯一标识符，通常是线程的 ID 或进程的 ID。
- **计数器**：记录当前持有锁的次数。
- **过期时间**：为了避免死锁，锁需要设定一个合理的超时时间。

我们可以在 Redis 中使用一个哈希表或简单的键值对来存储锁的状态。使用一个 key（如 `lock:<resource>`）来表示锁，包含以下字段：

- `owner`：当前持有锁的线程/进程标识符
- `count`：当前计数器，表示获得锁的次数
- `expires_at`：锁的到期时间，用于防止死锁

示例数据结构

```
{
  "key": "lock:resource_1",
  "value": {
    "owner": "thread_id_or_process_id",
    "count": 3,
    "expires_at": "2023-10-01T12:00:00Z"
  }
}
```

以下是围绕这个锁结构的基本操作：

1. 获取锁 (Lock Acquisition)：尝试设置 `lock:` 的值，如果这个 key 不存在（即没有锁），那么可以创建该 key，并设置 `owner` 为当前线程/进程的唯一 ID，`count` 设为 1，`expires_at` 设为当前时间加上锁的过期时间。如果该 key 已存在且 `owner` 是当前线程/进程的 ID，则增加 `count` 并更新 `expires_at`。
2. 释放锁 (Lock Release)：检查当前的 `owner` 是否是当前线程/进程的 ID。如果是，减少 `count`。如果 `count` 减少到 0，则删除该 key。如果在持有锁的情况下，锁已过期，系统会根据 `expires_at` 检查锁是否可释放，避免死锁。
3. 锁续期 (Lock Renewal)：如果当前线程在执行过程中需要继续持有锁，可以在逻辑处理中重新设置 `expires_at`。
4. 超时与故障恢复：可以设置锁的自动超时时间，例如，设定一个最大持锁时间，超时后锁自动释放。这可以在一定程度上防止死锁的情况。

整体流程示例：

- **获取锁：**
 - 线程A调用获取锁的 API。如果成功，返回锁的状态。
 - 如果线程B也尝试获取同一把锁，则会返回锁已被占用。

- **释放锁：**
 - 线程A在完成任务时调用释放锁的 API，递减 `count` 字段。
 - 如果 `count` 达到 0，删除锁并释放资源。
- **续约锁：**
 - 线程A在处理过程中可以根据需要续约锁，更新 `expires_at`。

你有看过一些负载均衡的一些方案吗

- **硬件负载均衡：**使用专用的硬件设备（如负载均衡器）来分配流量，比如F5设备，优点是性能强大，支持高并发。缺点是成本太高，通常一个专业级的硬件设备，都需要百万级别的价格。
- **软件负载均衡：**使用软件应用程序来实现负载均衡，可以运行在普通服务器上，比如 **Nginx** 支持反向代理和负载均衡，优点灵活性高，成本低，易于修改和扩展，缺点是性能不如硬件负载均衡，但是软件负载均衡的性能也足够应对大多数场景的并发量了。
- **DNS 负载均衡：**通过 DNS 服务器将流量分配到多个服务器上，不同的客户端请求可能获得不同的 IP 地址。优点是简单易用，不需要额外的硬件或软件。缺点是无法智能感知服务器的实时状态，缓存问题可能导致不均匀负载。
- **内容分发网络 (CDN)：**CDN 是一种分布式的网络结构，可以将内容分发到离用户最近的节点上。优点减少延迟，提高用户体验。缺点主要适用于静态内容，动态请求仍需其他形式的负载均衡。